Python Classes: Objects, Functions and Methods

BSF Training Programme: Introduction to Python (Think Python, Chapters 15–17)

Ramzan Basheer, SC22D036



Indian Institute of Space Science and Technology
Thiruvananthapuram, Kerala, India 695547

October 1, 2026

Outline

1. Getting Started
2. Classes and Objects
3. Classes and Functions
4. Classes and Methods
5. Worked Example and Wrap-up

Getting Started

Our 90 Minutes Today



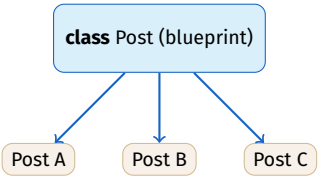
Time	Topic	Book
00–05	Why classes? The blueprint idea	–
05–25	Classes and objects: Point, Rectangle, copies	Ch. 15
25–40	Classes and functions: Time, pure functions, modifiers	Ch. 16
40–50	**Exercise 1** (hands-on)	Ch. 15–16
50–75	Classes and methods: `self`, `__init__`, `__str__`, `+`	Ch. 17
75–85	Worked example (Circle) and **Exercise 2**	Ch. 15–17
85–90	Recap and questions	–

Code in these slides uses Python 3. Every code box is a complete program: type it in and run it. Text after # is a comment: Python ignores it. -> in a comment shows the output you will see.

Why Classes? Blueprint vs Object



- A **class** is a blueprint. It says what every item of that kind has.
- An **object** is one real item built from the blueprint.
- One blueprint, many objects.
- Example: the standard design of a border post is the class. Each post on the ground is an object.
- Another: a blank personnel record form is the class. Each filled form is an object.



objects (also called **instances**)

Classes and Objects

Defining a Class and Making an Object



```
class Point:                          # create a new blueprint (class) named Point
    """A point on a map: x and y."""  # class body: a note for humans (docstring)

blank = Point()                       # use the blueprint once: make one object
print(blank)                          # show it -> <__main__.Point object at 0x...>
print(type(blank))                    # its type -> <class '__main__.Point'>

class Empty:                          # a class with nothing inside yet...
    pass                              # 'pass' = do nothing; fills the empty body
```

In plain words

`class Point:` makes the rubber stamp. `Point()` presses it once: one impression, one **object**. The object is an **instance** of `Point`. Making it is called **instantiation**.

A class body cannot be empty

`class Empty:` with nothing indented below it stops with `IndentationError: expected an indented block`. A comment alone is not enough. Put a docstring or `pass` inside.

Class and Object: Line by Line (1/2)



Line	Level	Code	What it does
1	**0**	`class Point:`	`class` starts a new blueprint. `Point` is its name. The colon means “the body of the class follows, indented below”.
2	**1**	`"""A point on a map: x and y."""`	The body. Text in triple quotes is a docstring: a note for humans. It also counts as the body, so the class is complete.
4	**0**	`blank = Point()`	`Point()` uses the blueprint once and makes one object. `=` stores that object under the name `blank`.
5	**0**	`print(blank)`	`print` shows the object. Without more work Python can only show its type and its memory address (`0x...`), which differs on every run.

Line = number on the code box. **Level** = indentation: 0 = at the left edge, 1 = 4 spaces in, 2 = 8, 3 = 12; cont. = continues the line above. Grey rows were explained on an earlier slide.

Class and Object: Line by Line (2/2)



Line	Level	Code	What it does
6	**0**	`print(type(blank))`	`type(...)` asks what kind of thing `blank` is. The answer is the class `Point`.
8	**0**	`class Empty:`	A second class, `Empty`, with nothing useful to put inside yet.
9	**1**	`    pass`	`pass` means “do nothing”. It fills the body. Without it (or a docstring), Python stops with `IndentationError`.

Line = number on the code box. **Level** = indentation: 0 = at the left edge, 1 = 4 spaces in, 2 = 8, 3 = 12; cont. = continues the line above. Grey rows were explained on an earlier slide.

Attributes: Named Values Inside an Object



```python
import math                                 # load Python's maths toolbox (for sqrt)

class Point:                                # blueprint for a point on a map
    """A point on a map: x and y."""        # class body (docstring)

blank = Point()                             # make one empty Point object
blank.x = 3.0                               # store 3.0 in the field x of blank
blank.y = 4.0                               # store 4.0 in the field y of blank
print(blank.y)                              # read field y back -> 4.0

d = math.sqrt(blank.x**2 + blank.y**2)      # distance from (0, 0); ** means power
print(d)                                    # -> 5.0

def print_point(p):                         # a function that receives a Point p
    print('(%g, %g)' % (p.x, p.y))          # print x and y; %g drops extra zeros

print_point(blank)                          # hand our object to the function -> (3, 4)
```

Common error

Asking for a field that was never stored, e.g. `blank.z`, stops with `AttributeError`. Check first with `hasattr(blank, 'z')`, which gives `True` or `False`.

Attributes: Line by Line (1/2)



Line	Level	Code	What it does
1	**0**	`import math`	`import` loads a toolbox that comes with Python. `math` holds maths helpers such as `sqrt` (square root).
3	**0**	`class Point:`	Start the Point blueprint.
4	**1**	`    """A point on a map: x and y."""`	Its body: a docstring.
6	**0**	`blank = Point()`	Make one empty Point object and call it `blank`.
7	**0**	`blank.x = 3.0`	Store the number 3.0 in a field named `x` inside `blank`. The field is created the moment we store into it. A field is called an attribute.
8	**0**	`blank.y = 4.0`	Store 4.0 in the field `y`.
9	**0**	`print(blank.y)`	Read the field `y` back and show it: `4.0`.

Line = number on the code box. **Level** = indentation: 0 = at the left edge, 1 = 4 spaces in, 2 = 8, 3 = 12; cont. = continues the line above. Grey rows were explained on an earlier slide.

Attributes: Line by Line (2/2)



Line	Level	Code	What it does
11	**0**	`d = math.sqrt(blank.x**2 + blank.y**2)`	`**2` means “squared”. This is the school distance formula from (0, 0) to (3, 4): square root of 9 + 16.
12	**0**	`print(d)`	Show the distance: `5.0`.
14	**0**	`def print_point(p):`	`def` defines a function named `print_point`. It will receive one Point, called `p` inside the function.
15	**1**	`    print('(%g, %g)' % (p.x, p.y))`	Show `x` and `y` in brackets. `%g` is a placeholder that prints a number without extra zeros; `%` fills the placeholders.
17	**0**	`print_point(blank)`	Hand our object to the function. Output: `(3, 4)`.

Line = number on the code box. **Level** = indentation: 0 = at the left edge, 1 = 4 spaces in, 2 = 8, 3 = 12; cont. = continues the line above. Grey rows were explained on an earlier slide.

Rectangle: An Object Inside an Object



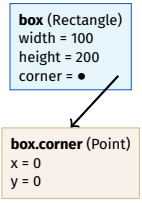
```python
class Point:                                # blueprint for a point
    """A point: x and y."""                 # class body (docstring)

class Rectangle:                            # blueprint for a rectangle
    """width, height and corner."""         # class body (docstring)

box = Rectangle()                           # make one empty Rectangle
box.width = 100.0                           # store its width
box.height = 200.0                          # store its height
box.corner = Point()                        # store a whole Point inside the box
box.corner.x = 0.0                          # x of the corner inside box
box.corner.y = 0.0                          # y of the corner inside box

def find_center(rect):                      # function: find the middle of rect
    p = Point()                             # make a new, empty Point
    p.x = rect.corner.x + rect.width / 2    # corner x plus half the width
    p.y = rect.corner.y + rect.height / 2 # corner y plus half the height
    return p                                # hand the new Point back

center = find_center(box)                   # center is the Point that came back
print(center.x, center.y)                   # -> 50.0 100.0
```



A Point stored inside another object is **embedded**. Read `box.corner.x` left to right: box, then its corner, then x.

Rectangle: Line by Line (1/2)



Line	Level	Code	What it does
1	0	`class Point:`	Start the Point blueprint (the same as before; every program here is complete on its own).
2	1	`    """A point: x and y."""`	Its body: a docstring.
4	0	`class Rectangle:`	Start a second blueprint, `Rectangle`.
5	1	`    """width, height and corner."""`	Its body: a docstring listing the fields we plan to store.
7	0	`box = Rectangle()`	Make one empty Rectangle called `box`.
8	0	`box.width = 100.0`	Store its width, 100.
9	0	`box.height = 200.0`	Store its height, 200.
10	0	`box.corner = Point()`	Make a new Point and store it inside `box` as the field `corner`. An object stored inside another object is called embedded.
11	0	`box.corner.x = 0.0`	Go to `box`, then to its `corner`, then set that corner's `x` to 0.

Line = number on the code box. **Level** = indentation: 0 = at the left edge, 1 = 4 spaces in, 2 = 8, 3 = 12; cont. = continues the line above. Grey rows were explained on an earlier slide.

Rectangle: Line by Line (2/2)



Line	Level	Code	What it does
12	**0**	`box.corner.y = 0.0`	Same for `y`.
14	**0**	`def find_center(rect):`	Define a function `find_center` that receives a rectangle, called `rect` inside.
15	**1**	`    p = Point()`	Make a new, empty Point `p` for the answer.
16	**1**	`    p.x = rect.corner.x + rect.width / 2`	Its `x`: the corner's x plus half the width (`/` means divide).
17	**1**	`    p.y = rect.corner.y + rect.height / 2`	Its `y`: the corner's y plus half the height.
18	**1**	`    return p`	`return` hands the new Point back to whoever called the function.
20	**0**	`center = find_center(box)`	Call the function with `box`. The Point that comes back is stored as `center`.
21	**0**	`print(center.x, center.y)`	Show both fields: `50.0 100.0`.

Line = number on the code box. **Level** = indentation: 0 = at the left edge, 1 = 4 spaces in, 2 = 8, 3 = 12; cont. = continues the line above. Grey rows were explained on an earlier slide.

Objects Are Mutable (They Can Change)



```
class Rectangle:                          # blueprint for a rectangle
    """width and height."""               # class body (docstring)

box = Rectangle()                         # make one rectangle
box.width = 100.0                         # its width
box.height = 200.0                        # its height

def grow_rectangle(rect, dwidth, dheight):   # rect will be ANOTHER NAME for box
    rect.width += dwidth                  # += means "add to": width grows
    rect.height += dheight                # height grows

print(box.width, box.height)              # before -> 100.0 200.0
grow_rectangle(box, 50, 100)              # hand box to the function
print(box.width, box.height)              # after  -> 150.0 300.0 (box changed)
```

In plain words

Mutable = can be changed after it is made. Inside the function, `rect` and `box` are two names for the **same** object. This is called **aliasing**.

Analogy

One duty register, two people holding it. If one writes in it, the other sees the new entry.

Mutable Objects: Line by Line (1/2)



Line	Level	Code	What it does
1	0	`class Rectangle:`	Start the Rectangle blueprint.
2	1	`"""width and height."""`	Its body: a docstring.
4	0	`box = Rectangle()`	Make one rectangle, `box`.
5	0	`box.width = 100.0`	Store its width, 100.
6	0	`box.height = 200.0`	Store its height, 200.
8	0	`def grow_rectangle(rect, dwidth, dheight):`	Define `grow_rectangle`. It receives a rectangle (`rect`) and two numbers. When we call it with `box`, `rect` becomes a second name for the same box.
9	1	`rect.width += dwidth`	`+=` means “add to”: the width grows by `dwidth`.

Line = number on the code box. **Level** = indentation: 0 = at the left edge, 1 = 4 spaces in, 2 = 8, 3 = 12; cont. = continues the line above. Grey rows were explained on an earlier slide.

Mutable Objects: Line by Line (2/2)



Line	Level	Code	What it does
10	**1**	`    rect.height += dheight`	The height grows by `dheight`.
12	**0**	`print(box.width, box.height)`	Show the size before: `100.0 200.0`.
13	**0**	`grow_rectangle(box, 50, 100)`	Call the function: `box` goes in, and inside it is called `rect`.
14	**0**	`print(box.width, box.height)`	Show the size after: `150.0 300.0`. The box itself changed, because `rect` was the same object. Two names for one object is called aliasing.

Line = number on the code box. **Level** = indentation: 0 = at the left edge, 1 = 4 spaces in, 2 = 8, 3 = 12; cont. = continues the line above. Grey rows were explained on an earlier slide.

Copying: Shallow vs Deep



```python
import copy                              # load Python's copying toolbox

class Point:                             # blueprint for a point
    """A point: x and y."""              # class body (docstring)

class Rectangle:                         # blueprint for a rectangle
    """width, height and corner."""      # class body (docstring)

p1 = Point()                             # make a Point...
p1.x = 3.0                               # ...set its x
p1.y = 4.0                               # ...set its y
p2 = copy.copy(p1)                       # p2 = a new Point with the same values
print(p1 is p2)                          # same object? -> False (two objects)

box = Rectangle()                        # make a Rectangle
box.width = 100.0                        # its width
box.height = 200.0                       # its height
box.corner = p1                          # its corner is a Point inside it

box2 = copy.copy(box)                    # SHALLOW copy: new box, SAME corner
print(box2.corner is box.corner)         # -> True  (the corner is shared)
box3 = copy.deepcopy(box)                # DEEP copy: new box AND new corner
print(box3.corner is box.corner)         # -> False (nothing is shared)
```

`is` asks: are these the very same object?

Analogy

Shallow copy = photocopy of a note saying “see locker 12”. Two notes, still one locker.

Deep copy = copy the note *and* the locker contents.

Copying: Line by Line (1/2)



Line	Level	Code	What it does
1	**0**	`import copy`	Load Python's `copy` toolbox.
3	**0**	`class Point:`	Start the Point blueprint.
4	**1**	`"""A point: x and y."""`	Its body: a docstring.
6	**0**	`class Rectangle:`	Start the Rectangle blueprint.
7	**1**	`"""width, height and corner."""`	Its body: a docstring.
9	**0**	`p1 = Point()`	Make a Point, `p1`.
10	**0**	`p1.x = 3.0`	Set its `x`.
11	**0**	`p1.y = 4.0`	Set its `y`.
12	**0**	`p2 = copy.copy(p1)`	`copy.copy` makes a new Point with the same values. `p2` is a separate object.
13	**0**	`print(p1 is p2)`	`is` asks: are these the very same object? `False`: two objects.
15	**0**	`box = Rectangle()`	Make a Rectangle, `box`.

Line = number on the code box. **Level** = indentation: 0 = at the left edge, 1 = 4 spaces in, 2 = 8, 3 = 12; cont. = continues the line above. Grey rows were explained on an earlier slide.

Copying: Line by Line (2/2)



Line	Level	Code	What it does
16	0	`box.width = 100.0`	Set its width.
17	0	`box.height = 200.0`	Set its height.
18	0	`box.corner = p1`	Store the Point `p1` inside it as its corner.
20	0	`box2 = copy.copy(box)`	Shallow copy: a new rectangle, but its `corner` field points to the same Point as `box`'s.
21	0	`print(box2.corner is box.corner)`	`True`: both rectangles share one corner. Moving one corner moves both.
22	0	`box3 = copy.deepcopy(box)`	Deep copy: a new rectangle and a new copy of everything inside it, corner included.
23	0	`print(box3.corner is box.corner)`	`False`: nothing is shared.

Line = number on the code box. **Level** = indentation: 0 = at the left edge, 1 = 4 spaces in, 2 = 8, 3 = 12; cont. = continues the line above. Grey rows were explained on an earlier slide.

Classes and Functions

The Time Class and Pure Functions



```
class Time:                                 # blueprint for a time of day
    """hour, minute and second."""          # class body (docstring)

def print_time(t):                          # print a Time as hh:mm:ss
    print('%.2d:%.2d:%.2d' %               # %.2d = a number with 2 digits...
          (t.hour, t.minute, t.second))     # ...filled with hour, minute, second

def add_time(t1, t2):                       # PURE function: builds a new answer
    total = Time()                          # a new, empty Time for the answer
    total.hour = t1.hour + t2.hour          # add the hours
    total.minute = t1.minute + t2.minute    # add the minutes
    total.second = t1.second + t2.second    # add the seconds
    if total.second >= 60:                  # 60 seconds or more?
        total.second -= 60                  #   take 60 seconds away
        total.minute += 1                   #   and carry 1 minute
    if total.minute >= 60:                  # 60 minutes or more?
        total.minute -= 60                  #   take 60 minutes away
        total.hour += 1                     #   and carry 1 hour
    return total                            # hand back the new Time

start = Time()                              # shift start time
start.hour, start.minute, start.second = 9, 45, 0         # = 09:45:00
duration = Time()                           # shift length
duration.hour, duration.minute, duration.second = 1, 35, 0  # = 01:35:00
print_time(add_time(start, duration))       # -> 11:20:00
print_time(start)                           # -> 09:45:00 (start is unchanged)
```

A **pure function** does not change its inputs. It builds and returns a **new** object.

Analogy

A clerk reads two reports and writes a fresh summary. The originals are untouched.

Pure Functions: Line by Line (1/3)



Line	Level	Code	What it does
1	**0**	`class Time:`	Start the Time blueprint.
2	**1**	`"""hour, minute and second."""`	Its body: a docstring.
4	**0**	`def print_time(t):`	Define `print_time`, which receives one Time, `t`.
5	**1**	`print('%.2d:%.2d:%.2d' %`	Print a pattern of three placeholders. `%.2d` means “a whole number, at least 2 digits”, so 9 prints as 09. The line continues below.
6	cont.	`(t.hour, t.minute, t.second))`	The three values that fill the placeholders: hour, minute, second.
8	**0**	`def add_time(t1, t2):`	Define `add_time`. It receives two Times. It is a pure function: it will not change them.
9	**1**	`total = Time()`	Make a new, empty Time `total` for the answer.

Line = number on the code box. **Level** = indentation: 0 = at the left edge, 1 = 4 spaces in, 2 = 8, 3 = 12; cont. = continues the line above. Grey rows were explained on an earlier slide.

Pure Functions: Line by Line (2/3)



Line	Level	Code	What it does
10	1	`total.hour = t1.hour + t2.hour`	Its hour: the two hours added.
11	1	`total.minute = t1.minute + t2.minute`	Its minute: the two minutes added.
12	1	`total.second = t1.second + t2.second`	Its second: the two seconds added.
13	1	`if total.second >= 60:`	`if` checks a condition: are there 60 seconds or more?
14	2	`total.second -= 60`	If yes, take 60 seconds away (`-=` means “subtract from”)…
15	2	`total.minute += 1`	…and carry 1 minute, as in school addition, but with 60 instead of 10.
16	1	`if total.minute >= 60:`	Are there 60 minutes or more?
17	2	`total.minute -= 60`	If yes, take 60 minutes away…
18	2	`total.hour += 1`	…and carry 1 hour.

Line = number on the code box. **Level** = indentation: 0 = at the left edge, 1 = 4 spaces in, 2 = 8, 3 = 12; cont. = continues the line above. Grey rows were explained on an earlier slide.

Pure Functions: Line by Line (3/3)



Line	Level	Code	What it does
19	**1**	`    return total`	Hand back the new Time. The two input Times were only read, never changed.
21	**0**	`start = Time()`	Make a Time for the shift start.
22	**0**	`start.hour, start.minute, start.second = 9, 45, 0`	Set its three fields in one line: the values on the right go, in order, into the names on the left. 09:45:00.
23	**0**	`duration = Time()`	Make a Time for the shift length.
24	**0**	`duration.hour, duration.minute, duration.second = 1, 35, 0`	Set it to 01:35:00.
25	**0**	`print_time(add_time(start, duration))`	Add them and print the answer: `11:20:00`.
26	**0**	`print_time(start)`	Print the start again: still `09:45:00`. Nothing was changed.

Line = number on the code box. **Level** = indentation: 0 = at the left edge, 1 = 4 spaces in, 2 = 8, 3 = 12; cont. = continues the line above. Grey rows were explained on an earlier slide.

Modifiers: Functions That Change the Object



```python
class Time:                                 # blueprint for a time of day
    """hour, minute and second."""          # class body (docstring)

def increment(t, seconds):                  # MODIFIER: changes t itself
    t.second += seconds                     # add the seconds into t
    if t.second >= 60:                      # 60 seconds or more?
        t.second -= 60                      #   take 60 seconds away
        t.minute += 1                       #   carry 1 minute
    if t.minute >= 60:                      # 60 minutes or more?
        t.minute -= 60                      #   take 60 minutes away
        t.hour += 1                         #   carry 1 hour
    # no return line: the result is the change inside t

start = Time()                              # make a Time...
start.hour, start.minute, start.second = 9, 45, 0  # ...09:45:00
increment(start, 30)                        # add 30 seconds
print(start.hour, start.minute, start.second)  # -> 9 45 30 (right)
increment(start, 150)                       # add 150 seconds
print(start.hour, start.minute, start.second)  # -> 9 46 120 (wrong!)
```

A **modifier** changes the object it is given. The caller sees the change.

The last line shows a bug: 120 seconds is not a real time. One carry is not enough when many seconds are added.

Which one to use?

Pure functions are easier to check. Think Python: write pure functions when you can, modifiers only when there is a clear benefit.

Modifiers: Line by Line (1/2)



Line	Level	Code	What it does
1	0	`class Time:`	Start the Time blueprint.
2	1	`"""hour, minute and second."""`	Its body: a docstring.
4	0	`def increment(t, seconds):`	Define `increment`. It receives a Time and a number of seconds. It is a modifier: it changes the Time it receives.
5	1	`t.second += seconds`	Add the seconds straight into `t`.
6	1	`if t.second >= 60:`	60 seconds or more?
7	2	`t.second -= 60`	Take 60 away…
8	2	`t.minute += 1`	…and carry 1 minute.
9	1	`if t.minute >= 60:`	60 minutes or more?
10	2	`t.minute -= 60`	Take 60 away…
11	2	`t.hour += 1`	…and carry 1 hour.
12	1	`# no return line: the result is the change inside t`	A comment line only. The function has no `return`: its result is the change it made inside `t`.

Line = number on the code box. **Level** = indentation: 0 = at the left edge, 1 = 4 spaces in, 2 = 8, 3 = 12; cont. = continues the line above. Grey rows were explained on an earlier slide.

Modifiers: Line by Line (2/2)



Line	Level	Code	What it does
14	0	`start = Time()`	Make a Time.
15	0	`start.hour, start.minute, start.second = 9, 45, 0`	Set it to 09:45:00.
16	0	`increment(start, 30)`	Add 30 seconds. `start` itself changes.
17	0	`print(start.hour, start.minute, start.second)`	Show it: `9 45 30`. Correct.
18	0	`increment(start, 150)`	Add 150 seconds.
19	0	`print(start.hour, start.minute, start.second)`	Show it: `9 46 120`. Wrong: 180 seconds minus one carry of 60 leaves 120. One carry is not enough for large numbers.

Line = number on the code box. **Level** = indentation: 0 = at the left edge, 1 = 4 spaces in, 2 = 8, 3 = 12; cont. = continues the line above. Grey rows were explained on an earlier slide.

Prototyping vs Planning



```python
class Time:                                 # blueprint for a time of day
    """hour, minute and second."""          # class body (docstring)

def time_to_int(t):                         # Time -> seconds since midnight
    minutes = t.hour * 60 + t.minute        # whole minutes so far
    return minutes * 60 + t.second          # whole seconds so far

def int_to_time(seconds):                   # seconds since midnight -> Time
    t = Time()                              # a new, empty Time
    minutes, t.second = divmod(seconds, 60) # divmod(130, 60) gives 2 and 10
    t.hour, t.minute = divmod(minutes, 60)  # split minutes into hours, minutes
    return t                                # hand back the new Time

def add_time(t1, t2):                       # PURE add: now only two lines
    total = time_to_int(t1) + time_to_int(t2)   # add two plain numbers
    return int_to_time(total)               # turn the total back into a Time

start = int_to_time(35100)                  # 35100 s after midnight = 09:45:00
answer = add_time(start, int_to_time(5700))  # add 5700 s = 1 h 35 min
print(answer.hour, answer.minute, answer.second)   # -> 11 20 0
```

Prototype and patch: write a rough version, test it, fix errors one by one.

Planned development: think first. A time of day is just a number of seconds since midnight.

`divmod(a, b)` gives the quotient and the remainder together.

Planning: Line by Line (1/2)



Line	Level	Code	What it does
1	0	`class Time:`	Start the Time blueprint.
2	1	`"""hour, minute and second."""`	Its body: a docstring.
4	0	`def time_to_int(t):`	Define `time_to_int`: turn a Time into one number, the seconds since midnight.
5	1	`minutes = t.hour * 60 + t.minute`	Hours times 60, plus minutes: the whole minutes since midnight.
6	1	`return minutes * 60 + t.second`	Minutes times 60, plus seconds: the whole seconds. `return` hands the number back.
8	0	`def int_to_time(seconds):`	Define `int_to_time`: the reverse, a number of seconds back into a Time.
9	1	`t = Time()`	Make a new, empty Time.
10	1	`minutes, t.second = divmod(seconds, 60)`	`divmod(a, b)` gives two answers: how many whole times b fits into a, and what is left over. `divmod(130, 60)` gives 2 and 10. The two answers go into `minutes` and `t.second`.

Line = number on the code box. **Level** = indentation: 0 = at the left edge, 1 = 4 spaces in, 2 = 8, 3 = 12; cont. = continues the line above. Grey rows were explained on an earlier slide.

Planning: Line by Line (2/2)



Line	Level	Code	What it does
11	1	`t.hour, t.minute = divmod(minutes, 60)`	Split the minutes the same way into hours and minutes.
12	1	`return t`	Hand back the new Time.
14	0	`def add_time(t1, t2):`	The new `add_time`, still a pure function.
15	1	`total = time_to_int(t1) + time_to_int(t2)`	Turn both Times into seconds and add two plain numbers.
16	1	`return int_to_time(total)`	Turn the total back into a Time and hand it back. No carry rules needed.
18	0	`start = int_to_time(35100)`	Make 09:45:00 from its seconds: 9 x 3600 + 45 x 60 = 35100.
19	0	`answer = add_time(start, int_to_time(5700))`	Add 5700 seconds (1 h 35 min) using `add_time`.
20	0	`print(answer.hour, answer.minute, answer.second)`	Show the answer: `11 20 0`.

Line = number on the code box. **Level** = indentation: 0 = at the left edge, 1 = 4 spaces in, 2 = 8, 3 = 12; cont. = continues the line above. Grey rows were explained on an earlier slide.

Exercise 1 (10 minutes)



1. Write a **modifier** `move_rectangle(rect, dx, dy)` that shifts the corner of a Rectangle by `dx` and `dy`.
2. Write a **pure** version `moved_rectangle(rect, dx, dy)` that returns a new Rectangle and leaves the old one unchanged. Hint: `copy.deepcopy`.

Starter code: type this first, then add your two functions below it.

```
import copy                             # copying toolbox (needed for task 2)

class Point:                            # blueprint for a point
    """A point: x and y."""             # class body (docstring)

class Rectangle:                        # blueprint for a rectangle
    """width, height and corner."""     # class body (docstring)

box = Rectangle()                       # a rectangle to practise on
box.width, box.height = 100.0, 200.0    # its size (two values in one line)
box.corner = Point()                    # its corner is a Point...
box.corner.x, box.corner.y = 0.0, 0.0   # ...placed at (0, 0)
```

Exercise 1 Starter Code: Line by Line



Line	Level	Code	What it does
1	0	`import copy`	Load the copy toolbox; task 2 needs `copy.deepcopy`.
3	0	`class Point:`	Start the Point blueprint.
4	1	`"""A point: x and y."""`	Its body: a docstring.
6	0	`class Rectangle:`	Start the Rectangle blueprint.
7	1	`"""width, height and corner."""`	Its body: a docstring.
9	0	`box = Rectangle()`	Make a rectangle to practise on.
10	0	`box.width, box.height = 100.0, 200.0`	Set width and height in one line: 100 and 200.
11	0	`box.corner = Point()`	Store a new Point inside it as its corner.
12	0	`box.corner.x, box.corner.y = 0.0, 0.0`	Put that corner at (0, 0).

Line = number on the code box. **Level** = indentation: 0 = at the left edge, 1 = 4 spaces in, 2 = 8, 3 = 12; cont. = continues the line above. Grey rows were explained on an earlier slide.

Exercise 1: One Possible Solution



The starter code plus the two functions, as one complete program.

```
import copy                             # copying toolbox (needed for task 2)

class Point:                            # blueprint for a point
    """A point: x and y."""             # class body (docstring)

class Rectangle:                        # blueprint for a rectangle
    """width, height and corner."""     # class body (docstring)

box = Rectangle()                       # a rectangle to practise on
box.width, box.height = 100.0, 200.0    # its size (two values in one line)
box.corner = Point()                    # its corner is a Point...
box.corner.x, box.corner.y = 0.0, 0.0   # ...placed at (0, 0)
def move_rectangle(rect, dx, dy):       # MODIFIER: moves rect itself
    rect.corner.x += dx                 # shift the corner right by dx
    rect.corner.y += dy                 # shift the corner up by dy

def moved_rectangle(rect, dx, dy):      # PURE: gives back a moved copy
    new = copy.deepcopy(rect)           # full copy, corner included
    move_rectangle(new, dx, dy)         # move the copy only
    return new                          # hand the copy back

b2 = moved_rectangle(box, 5, 5)         # moved copy of box
print(box.corner.x, b2.corner.x)        # -> 0.0 5.0  (original stayed put)
move_rectangle(box, 10, 20)             # now move the original itself
print(box.corner.x, box.corner.y)       # -> 10.0 20.0
```

Exercise 1 Solution: Line by Line (1/2)



Line	Level	Code	What it does
1	0	`import copy`	Load the copy toolbox; task 2 needs `copy.deepcopy`.
3	0	`class Point:`	Start the Point blueprint.
4	1	`    """A point: x and y."""`	Its body: a docstring.
6	0	`class Rectangle:`	Start the Rectangle blueprint.
7	1	`    """width, height and corner."""`	Its body: a docstring.
9	0	`box = Rectangle()`	Make a rectangle to practise on.
10	0	`box.width, box.height = 100.0, 200.0`	Set width and height in one line: 100 and 200.
11	0	`box.corner = Point()`	Store a new Point inside it as its corner.
12	0	`box.corner.x, box.corner.y = 0.0, 0.0`	Put that corner at (0, 0).
13	0	`def move_rectangle(rect, dx, dy):`	Task 1. Define the modifier `move_rectangle`: a rectangle and two distances.
14	1	`    rect.corner.x += dx`	Move the corner right by `dx`. The rectangle itself changes.

Line = number on the code box. **Level** = indentation: 0 = at the left edge, 1 = 4 spaces in, 2 = 8, 3 = 12; cont. = continues the line above. Grey rows were explained on an earlier slide.

Exercise 1 Solution: Line by Line (2/2)



Line	Level	Code	What it does
15	**1**	`    rect.corner.y += dy`	Move the corner up by `dy`.
17	**0**	`def moved_rectangle(rect, dx, dy):`	Task 2. Define the pure function `moved_rectangle`.
18	**1**	`    new = copy.deepcopy(rect)`	Make a deep copy, so the corner is copied too.
19	**1**	`    move_rectangle(new, dx, dy)`	Reuse task 1 on the copy only.
20	**1**	`    return new`	Hand back the moved copy.
22	**0**	`b2 = moved_rectangle(box, 5, 5)`	Test task 2: `b2` is a moved copy of `box`.
23	**0**	`print(box.corner.x, b2.corner.x)`	Show both corners' x: `0.0 5.0`. The original did not move.
24	**0**	`move_rectangle(box, 10, 20)`	Test task 1: move the original itself.
25	**0**	`print(box.corner.x, box.corner.y)`	Show its corner: `10.0 20.0`.

Line = number on the code box. **Level** = indentation: 0 = at the left edge, 1 = 4 spaces in, 2 = 8, 3 = 12; cont. = continues the line above. Grey rows were explained on an earlier slide.

Classes and Methods

From Functions to Methods



```
class Time:                                     # blueprint for a time of day
    """hour, minute and second."""              # class body (docstring)
    def print_time(self):                       # a METHOD: a function inside the class
        print('%.2d:%.2d:%.2d' %                # hh:mm:ss, two digits each...
              (self.hour, self.minute, self.second))   # ...from MY own fields

start = Time()                                  # make a Time object
start.hour = 9                                  # set its hour
start.minute = 45                               # set its minute
start.second = 0                                # set its second
start.print_time()                              # "Time, print yourself" -> 09:45:00
Time.print_time(start)                          # the same call, written out -> 09:45:00
```

In plain words

A **method** is a function defined **inside** a class. Call it with a dot: `start.print_time()`. Every Time object built from the blueprint carries it.

Analogy

Function: “Clerk, print this time.”
Method: “Time, print yourself.”

Methods: Line by Line (1/2)



Line	Level	Code	What it does
1	0	`class Time:`	Start the Time blueprint.
2	1	`"""hour, minute and second."""`	Its docstring.
3	1	`def print_time(self):`	`def` indented inside the class: a function that belongs to the class. This is a method. Its first input, `self`, is the Time it is called on.
4	2	`print('%.2d:%.2d:%.2d' %`	Print hh:mm:ss (two digits each). The line continues below.
5	cont.	`(self.hour, self.minute, self.second))`	The values come from `self`: this Time's own hour, minute and second.

Line = number on the code box. **Level** = indentation: 0 = at the left edge, 1 = 4 spaces in, 2 = 8, 3 = 12; cont. = continues the line above. Grey rows were explained on an earlier slide.

Methods: Line by Line (2/2)



Line	Level	Code	What it does
7	0	`start = Time()`	Make a Time object, `start`.
8	0	`start.hour = 9`	Set its hour.
9	0	`start.minute = 45`	Set its minute.
10	0	`start.second = 0`	Set its second.
11	0	`start.print_time()`	Call the method with a dot: “start, print yourself”. Output `09:45:00`.
12	0	`Time.print_time(start)`	The same call written out in full: the method from the class, with `start` handed in as `self`. Same output.

Line = number on the code box. **Level** = indentation: 0 = at the left edge, 1 = 4 spaces in, 2 = 8, 3 = 12; cont. = continues the line above. Grey rows were explained on an earlier slide.

What Is self?



```
class Time:                                          # blueprint for a time of day
    """hour, minute and second."""                   # class body (docstring)
    def time_to_int(self):                           # self = the object before the dot
        minutes = self.hour * 60 + self.minute       # MY hours and minutes, as minutes
        return minutes * 60 + self.second            # MY time, as seconds

start = Time()                                       # first Time object
start.hour, start.minute, start.second = 9, 45, 0    # 09:45:00
lunch = Time()                                       # second Time object
lunch.hour, lunch.minute, lunch.second = 13, 0, 0    # 13:00:00

print(start.time_to_int())                           # self is start -> 35100
print(lunch.time_to_int())                           # self is lunch -> 46800
print(Time.time_to_int(start))                       # what Python really runs -> 35100
```

In plain words

`self` means “the object this method was called on”. `start.time_to_int()` is run as `Time.time_to_int(start)`: Python passes `start` in as `self` for you.

Common error

Leaving `self` out of the `def` line gives `TypeError: Time.time_to_int() takes 0 positional arguments but 1 was given.`

self: Line by Line (1/2)



Line	Level	Code	What it does
1	0	`class Time:`	Start the Time blueprint.
2	1	`"""hour, minute and second."""`	Its docstring.
3	1	`def time_to_int(self):`	A method. `self` will be whichever Time is written before the dot.
4	2	`minutes = self.hour * 60 + self.minute`	This Time's hours times 60 plus its minutes.
5	2	`return minutes * 60 + self.second`	Times 60 plus its seconds: the time as seconds since midnight.
7	0	`start = Time()`	Make a first Time, `start`.
8	0	`start.hour, start.minute, start.second = 9, 45, 0`	Set it to 09:45:00.

Line = number on the code box. **Level** = indentation: 0 = at the left edge, 1 = 4 spaces in, 2 = 8, 3 = 12; cont. = continues the line above. Grey rows were explained on an earlier slide.

self: Line by Line (2/2)



Line	Level	Code	What it does
9	**0**	`lunch = Time()`	Make a second Time, `lunch`.
10	**0**	`lunch.hour, lunch.minute, lunch.second = 13, 0, 0`	Set it to 13:00:00.
12	**0**	`print(start.time_to_int())`	Call the method on `start`: inside, `self` is `start`. Output `35100`.
13	**0**	`print(lunch.time_to_int())`	Same method on `lunch`: now `self` is `lunch`. Output `46800`.
14	**0**	`print(Time.time_to_int(start))`	What Python really runs for `start.time_to_int()`: the class's method with `start` as the first input. Output `35100`.

Line = number on the code box. **Level** = indentation: 0 = at the left edge, 1 = 4 spaces in, 2 = 8, 3 = 12; cont. = continues the line above. Grey rows were explained on an earlier slide.

The __init__ Method: Setting Up a New Object



```
class Time:                                          # blueprint for a time of day
    """hour, minute and second."""                   # class body (docstring)
    def __init__(self, hour=0, minute=0, second=0):  # runs by itself on Time(...)
        self.hour = hour                             # store hour in MY field
        self.minute = minute                         # store minute in MY field
        self.second = second                         # store second in MY field

t1 = Time()                                          # nothing given -> 0, 0, 0
t2 = Time(9)                                         # hour only     -> 9, 0, 0
t3 = Time(9, 45)                                     # hour, minute  -> 9, 45, 0
t4 = Time(9, 45, 30)                                 # all three     -> 9, 45, 30
print(t3.hour, t3.minute, t3.second)                 # -> 9 45 0
```

In plain words

`__init__` (“init” = initialise; two underscores on each side) runs **by itself** when an object is made. `hour=0` is a **default value**: used when you give nothing.

Analogy

Joining formalities: every new recruit gets name, number and unit recorded on day one.

__init__: Line by Line



Line	Level	Code	What it does
1	0	`class Time:`	Start the Time blueprint.
2	1	`"""hour, minute and second."""`	Its docstring.
3	1	`def __init__(self, hour=0, minute=0, second=0):`	`__init__` (two underscores each side) runs by itself every time we write `Time(...)`. `hour=0` is a default value: used when nothing is given.
4	2	`self.hour = hour`	Store the given hour in this new object's field `hour`.
5	2	`self.minute = minute`	Store the minute.
6	2	`self.second = second`	Store the second.
8	0	`t1 = Time()`	Nothing given: all defaults, 00:00:00.
9	0	`t2 = Time(9)`	One value: it goes to `hour`. 09:00:00.
10	0	`t3 = Time(9, 45)`	Two values: hour and minute. 09:45:00.
11	0	`t4 = Time(9, 45, 30)`	Three values. 09:45:30.
12	0	`print(t3.hour, t3.minute, t3.second)`	Show `t3`'s fields: `9 45 0`.

Line = number on the code box. **Level** = indentation: 0 = at the left edge, 1 = 4 spaces in, 2 = 8, 3 = 12; cont. = continues the line above. Grey rows were explained on an earlier slide.

The __str__ Method: How an Object Describes Itself



```
class Time:                                          # blueprint for a time of day
    """hour, minute and second."""                   # class body (docstring)
    def __init__(self, hour=0, minute=0, second=0):  # set up a new Time
        self.hour = hour                             # store hour
        self.minute = minute                         # store minute
        self.second = second                         # store second
    def __str__(self):                               # used by print() and str()
        return '%.2d:%.2d:%.2d' % (                  # build the text hh:mm:ss...
            self.hour, self.minute, self.second)     # ...from MY fields

t = Time(9, 45)                                      # make a Time
print(t)                                             # print uses __str__ -> 09:45:00
text = str(t)                                        # str() also uses __str__
print('Shift starts at ' + text)                     # -> Shift starts at 09:45:00
```

In plain words

Without `__str__`, `print(t)` shows an unreadable memory address. `__str__` must **return** the text; it should not print it. Tip: write `__init__` and `__str__` first for every new class.

Analogy

The name tag on the uniform.

__str__: Line by Line (1/2)



Line	Level	Code	What it does
1	0	`class Time:`	Start the Time blueprint.
2	1	`"""hour, minute and second."""`	Its docstring.
3	1	`def __init__(self, hour=0, minute=0, second=0):`	`__init__`, as on the previous slide: set up a new Time.
4	2	`self.hour = hour`	Store the hour.
5	2	`self.minute = minute`	Store the minute.
6	2	`self.second = second`	Store the second.
7	1	`def __str__(self):`	`__str__` is called by `print()` and `str()` when they need text for this object.
8	2	`return '%.2d:%.2d:%.2d' % (`	Build the text hh:mm:ss and `return` it (not print it). The line continues below.

Line = number on the code box. **Level** = indentation: 0 = at the left edge, 1 = 4 spaces in, 2 = 8, 3 = 12; cont. = continues the line above. Grey rows were explained on an earlier slide.

__str__: Line by Line (2/2)



Line	Level	Code	What it does
9	cont.	`self.hour, self.minute, self.second)`	The values: this Time's own fields.
11	**0**	`t = Time(9, 45)`	Make 09:45:00.
12	**0**	`print(t)`	`print` asks the object for its text through `__str__`. Output `09:45:00`.
13	**0**	`text = str(t)`	`str(t)` gets the same text and stores it as `text`. Text in Python is called a string.
14	**0**	`print('Shift starts at ' + text)`	`+` between two strings joins them. Output `Shift starts at 09:45:00`.

Line = number on the code box. **Level** = indentation: 0 = at the left edge, 1 = 4 spaces in, 2 = 8, 3 = 12; cont. = continues the line above. Grey rows were explained on an earlier slide.

Operator Overloading: Teaching + to Work on Time



```python
class Time:                                          # blueprint for a time of day
    def __init__(self, hour=0, minute=0, second=0):  # set up a new Time
        self.hour, self.minute, self.second = hour, minute, second  # store all three
    def __str__(self):                               # text for print()
        return '%.2d:%.2d:%.2d' % (self.hour, self.minute, self.second)  # hh:mm:ss
    def time_to_int(self):                           # MY time, in seconds
        return self.hour * 3600 + self.minute * 60 + self.second  # 1 h = 3600 s
    def __add__(self, other):                        # runs for: self + other
        if isinstance(other, Time):                  # is other a Time?
            other = other.time_to_int()              #   then use its seconds
        minutes, second = divmod(self.time_to_int() + other, 60)  # total -> min, s
        hour, minute = divmod(minutes, 60)           # minutes -> h, min
        return Time(hour, minute, second)            # a new Time for the sum
    def __radd__(self, other):                       # runs for: number + self
        return self.__add__(other)                   # same as self + number

start = Time(9, 45)                                  # 09:45:00
duration = Time(1, 35)                               # 01:35:00
print(start + duration)                              # Time + Time   -> 11:20:00
print(start + 1337)                                  # Time + number -> 10:07:17
print(1337 + start)                                  # number + Time -> 10:07:17
```

Writing `__add__` tells Python what `+` means for our class. This is **operator overloading**.

`isinstance` checks the type of `other`, then picks the right action (**type-based dispatch**).

`__radd__` handles a number on the **left** of `+`.

Operator Overloading: Line by Line (1/3)



Line	Level	Code	What it does
1	0	`class Time:`	Start the Time blueprint (complete, all methods in one place).
2	1	`def __init__(self, hour=0, minute=0, second=0):`	`__init__` with defaults: set up a new Time.
3	2	`self.hour, self.minute, self.second = hour, minute, second`	Store all three fields in one line; the values go in order.
4	1	`def __str__(self):`	`__str__`: the text for `print()`.
5	2	`return '%.2d:%.2d:%.2d' % (self.hour, self.minute, self.second)`	Return hh:mm:ss.
6	1	`def time_to_int(self):`	A helper method: this Time as seconds since midnight.
7	2	`return self.hour * 3600 + self.minute * 60 + self.second`	One hour is 3600 seconds, one minute 60.

Line = number on the code box. **Level** = indentation: 0 = at the left edge, 1 = 4 spaces in, 2 = 8, 3 = 12; cont. = continues the line above. Grey rows were explained on an earlier slide.

Operator Overloading: Line by Line (2/3)



Line	Level	Code	What it does
8	1	`def __add__(self, other):`	`__add__` runs whenever Python sees `self + other` with a Time on the left.
9	2	`if isinstance(other, Time):`	`isinstance` checks the type: is `other` a Time?
10	3	`other = other.time_to_int()`	If yes, replace it by its seconds, so from here `other` is always a number.
11	2	`minutes, second = divmod(self.time_to_int() + other, 60)`	Add the seconds, then `divmod` by 60 gives whole minutes and leftover seconds.
12	2	`hour, minute = divmod(minutes, 60)`	`divmod` by 60 again gives hours and leftover minutes.
13	2	`return Time(hour, minute, second)`	Build and return a new Time for the sum. The two inputs are unchanged.
14	1	`def __radd__(self, other):`	`__radd__` runs for `number + Time`, with the Time on the right.

Line = number on the code box. **Level** = indentation: 0 = at the left edge, 1 = 4 spaces in, 2 = 8, 3 = 12; cont. = continues the line above. Grey rows were explained on an earlier slide.

Operator Overloading: Line by Line (3/3)



Line	Level	Code	What it does
15	**2**	`        return self.__add__(other)`	Adding is the same either way round, so reuse `__add__`.
17	**0**	`start = Time(9, 45)`	Make 09:45:00.
18	**0**	`duration = Time(1, 35)`	Make 01:35:00.
19	**0**	`print(start + duration)`	Time + Time: Python runs `start.__add__(duration)`. Output `11:20:00`.
20	**0**	`print(start + 1337)`	Time + number of seconds. Output `10:07:17`.
21	**0**	`print(1337 + start)`	Number + Time: Python runs `start.__radd__(1337)`. Output `10:07:17`.

Line = number on the code box. **Level** = indentation: 0 = at the left edge, 1 = 4 spaces in, 2 = 8, 3 = 12; cont. = continues the line above. Grey rows were explained on an earlier slide.

Polymorphism: One Function, Many Types



```
class Time:                                          # same Time class as before
    def __init__(self, hour=0, minute=0, second=0):  # set up a new Time
        self.hour, self.minute, self.second = hour, minute, second  # store all three
    def __str__(self):                               # text for print()
        return '%.2d:%.2d:%.2d' % (self.hour, self.minute, self.second)  # hh:mm:ss
    def time_to_int(self):                           # MY time, in seconds
        return self.hour * 3600 + self.minute * 60 + self.second  # 1 h = 3600 s
    def __add__(self, other):                        # runs for: self + other
        if isinstance(other, Time):                  # is other a Time?
            other = other.time_to_int()              #   then use its seconds
        minutes, second = divmod(self.time_to_int() + other, 60)  # total -> min, s
        hour, minute = divmod(minutes, 60)           # minutes -> h, min
        return Time(hour, minute, second)            # a new Time for the sum
    def __radd__(self, other):                       # runs for: number + self
        return self.__add__(other)                   # same as self + number

print(sum([1, 2, 3]))                                # sum of whole numbers -> 6
print(sum([0.5, 0.25]))                              # sum of decimals      -> 0.75
duties = [Time(7, 43), Time(7, 41), Time(7, 37)]     # a list of three Times
print(sum(duties))                                   # sum of Times -> 23:01:00
```

A function that works with many types is **polymorphic**. Python's `sum` adds whole numbers, decimals and our Times. We never changed `sum`; Time only had to know `+`.

Analogy

One command, "Fall in!", works for every platoon.

Polymorphism: Line by Line (1/2)



Line	Level	Code	What it does
1	0	`class Time:`	The same complete Time class as on the operator overloading slide.
2	1	`    def __init__(self, hour=0, minute=0, second=0):`	`__init__`: set up a new Time.
3	2	`        self.hour, self.minute, self.second = hour, minute, second`	Store all three fields.
4	1	`    def __str__(self):`	`__str__`: text for `print()`.
5	2	`        return '%.2d:%.2d:%.2d' % (self.hour, self.minute, self.second)`	Return hh:mm:ss.
6	1	`    def time_to_int(self):`	Helper: this Time in seconds.
7	2	`        return self.hour * 3600 + self.minute * 60 + self.second`	Hours, minutes and seconds as one number.
8	1	`    def __add__(self, other):`	`__add__`: runs for `self + other`.
9	2	`        if isinstance(other, Time):`	Is `other` a Time?
10	3	`            other = other.time_to_int()`	If yes, use its seconds.

Line = number on the code box. **Level** = indentation: 0 = at the left edge, 1 = 4 spaces in, 2 = 8, 3 = 12; cont. = continues the line above. Grey rows were explained on an earlier slide.

Polymorphism: Line by Line (2/2)



Line	Level	Code	What it does
11	2	`minutes, second = divmod(self.time_to_int() + other, 60)`	Total seconds, split into minutes and seconds.
12	2	`hour, minute = divmod(minutes, 60)`	Minutes split into hours and minutes.
13	2	`return Time(hour, minute, second)`	Return a new Time for the sum.
14	1	`def __radd__(self, other):`	`__radd__`: runs for `number + self`.
15	2	`return self.__add__(other)`	Same as `self + number`.
17	0	`print(sum([1, 2, 3]))`	`sum` is built into Python. On whole numbers: `6`.
18	0	`print(sum([0.5, 0.25]))`	On decimal numbers: `0.75`.
19	0	`duties = [Time(7, 43), Time(7, 41), Time(7, 37)]`	A list (square brackets) of three Time objects.
20	0	`print(sum(duties))`	`sum` on Times: `23:01:00`. `sum` starts from 0 and uses `+` each step, so it calls our `__radd__` and `__add__`. We never changed `sum`.

Line = number on the code box. **Level** = indentation: 0 = at the left edge, 1 = 4 spaces in, 2 = 8, 3 = 12; cont. = continues the line above. Grey rows were explained on an earlier slide.

Interface vs Implementation



```
class Time:                                          # Version B: a new INSIDE
    """Stores one number: seconds since midnight."""  # class body (docstring)
    def __init__(self, hour=0, minute=0, second=0):  # same way to create a Time
        self.seconds = hour * 3600 + minute * 60 + second  # one field only
    def __str__(self):                               # same text for print()
        minutes, s = divmod(self.seconds, 60)        # seconds -> minutes, seconds
        h, m = divmod(minutes, 60)                   # minutes -> hours, minutes
        return '%.2d:%.2d:%.2d' % (h, m, s)          # hh:mm:ss, as before

t = Time(9, 45)                                      # used exactly as before...
print(t)                                             # ...same result -> 09:45:00
print(t.seconds)                                     # the new inside -> 35100
```

In plain words

Interface: what users of the class call: `Time(9, 45)`, `print`. **Implementation**: the fields and code inside. Keep the interface fixed and the inside can change safely.

Analogy

A driver uses steering, clutch and brake (interface). The engine can be petrol or diesel (implementation).

Interface vs Implementation: Line by Line (1/2)



Line	Level	Code	What it does
1	0	`class Time:`	Version B of Time: a different inside.
2	1	`"""Stores one number: seconds since midnight."""`	Its docstring: this version stores one number only.
3	1	`def __init__(self, hour=0, minute=0, second=0):`	Created exactly as before: `Time(hour, minute, second)`.
4	2	`self.seconds = hour * 3600 + minute * 60 + second`	Inside, it keeps one field, `seconds`, instead of three.
5	1	`def __str__(self):`	`__str__`: the same text as before.
6	2	`minutes, s = divmod(self.seconds, 60)`	Split the seconds into minutes and seconds.

Line = number on the code box. **Level** = indentation: 0 = at the left edge, 1 = 4 spaces in, 2 = 8, 3 = 12; cont. = continues the line above. Grey rows were explained on an earlier slide.

Interface vs Implementation: Line by Line (2/2)



Line	Level	Code	What it does
7	**2**	`h, m = divmod(minutes, 60)`	Split the minutes into hours and minutes.
8	**2**	`return '%.2d:%.2d:%.2d' % (h, m, s)`	Return hh:mm:ss.
10	**0**	`t = Time(9, 45)`	Users create a Time exactly as before (the interface did not change)…
11	**0**	`print(t)`	…and see exactly the same output: `09:45:00`.
12	**0**	`print(t.seconds)`	The inside is new: one field holding `35100`. Only the implementation changed.

Line = number on the code box. **Level** = indentation: 0 = at the left edge, 1 = 4 spaces in, 2 = 8, 3 = 12; cont. = continues the line above. Grey rows were explained on an earlier slide.

Worked Example and Wrap-up

Worked Example: Circle (Everything Together)



```python
import math                                          # maths toolbox (pi, sqrt)

class Point:                                         # a point on the map
    def __init__(self, x=0.0, y=0.0):                # set up x and y
        self.x, self.y = x, y                        # store both
    def __str__(self):                               # text for print()
        return '(%g, %g)' % (self.x, self.y)         # e.g. (150, 100)

class Circle:                                        # a circle on the map
    def __init__(self, center, radius):              # set up centre and radius
        self.center = center                         # a Point stored inside the Circle
        self.radius = radius                         # a plain number (metres)
    def __str__(self):                               # text for print()
        return 'Circle at %s, radius %g' % (self.center, self.radius)  # %s: Point's text
    def area(self):                                  # MY area
        return math.pi * self.radius ** 2            # pi x radius x radius
    def contains(self, p):                           # is Point p inside me?
        dx = p.x - self.center.x                     # left-right gap to the centre
        dy = p.y - self.center.y                     # up-down gap to the centre
        return math.sqrt(dx**2 + dy**2) <= self.radius  # within the radius?

post = Circle(Point(150, 100), 75)                   # watch radius 75 m around (150, 100)
print(post)                                          # -> Circle at (150, 100), radius 75
print(round(post.area(), 1))                         # -> 17671.5 (square metres)
print(post.contains(Point(160, 90)))                 # -> True (inside the radius)
```

In one program: classes with `__init__` and `__str__`; an embedded Point; methods using `self`.

Real use

`contains` answers: “Is this location inside a 75 m watch radius?”

Circle: Line by Line (1/3)



Line	Level	Code	What it does
1	0	`import math`	Load the maths toolbox for `pi` and `sqrt`.
3	0	`class Point:`	Start the Point blueprint, now with methods.
4	1	`    def __init__(self, x=0.0, y=0.0):`	`__init__`: set up x and y; both default to 0.
5	2	`        self.x, self.y = x, y`	Store both values in one line.
6	1	`    def __str__(self):`	`__str__`: text for `print()`.
7	2	`        return '(%g, %g)' % (self.x, self.y)`	Return `(x, y)`; `%g` drops extra zeros.
9	0	`class Circle:`	Start the Circle blueprint.
10	1	`    def __init__(self, center, radius):`	`__init__`: a Circle needs a centre and a radius.

Line = number on the code box. **Level** = indentation: 0 = at the left edge, 1 = 4 spaces in, 2 = 8, 3 = 12; cont. = continues the line above. Grey rows were explained on an earlier slide.

Circle: Line by Line (2/3)



Line	Level	Code	What it does
11	2	`self.center = center`	Store the centre, which is a Point: an embedded object.
12	2	`self.radius = radius`	Store the radius, a plain number (metres).
13	1	`def __str__(self):`	`__str__`: text for `print()`.
14	2	`return 'Circle at %s, radius %g' % (self.center, self.radius)`	`%s` inserts text; for the centre, Python uses the Point's own `__str__`.
15	1	`def area(self):`	Method `area`.
16	2	`return math.pi * self.radius ** 2`	pi times radius squared.
17	1	`def contains(self, p):`	Method `contains`: is Point `p` inside this circle?
18	2	`dx = p.x - self.center.x`	Left-right distance from the centre.

Line = number on the code box. **Level** = indentation: 0 = at the left edge, 1 = 4 spaces in, 2 = 8, 3 = 12; cont. = continues the line above. Grey rows were explained on an earlier slide.

Circle: Line by Line (3/3)



Line	Level	Code	What it does
19	**2**	`dy = p.y - self.center.y`	Up-down distance from the centre.
20	**2**	`return math.sqrt(dx**2 + dy**2) <= self.radius`	Straight-line distance (school formula), compared with the radius. `<=` gives `True` or `False`.
22	**0**	`post = Circle(Point(150, 100), 75)`	Make a circle: centre (150, 100), radius 75. Call it `post`.
23	**0**	`print(post)`	Its text: `Circle at (150, 100), radius 75`.
24	**0**	`print(round(post.area(), 1))`	Its area, rounded to 1 decimal place: `17671.5`.
25	**0**	`print(post.contains(Point(160, 90)))`	Is the location (160, 90) inside? `True`.

Line = number on the code box. **Level** = indentation: 0 = at the left edge, 1 = 4 spaces in, 2 = 8, 3 = 12; cont. = continues the line above. Grey rows were explained on an earlier slide.

Exercise 2 (7 minutes)



1. Rewrite the `Rectangle` class with an `__init__` that takes `corner`, `width` and `height`.
2. Add a `__str__` method and an `area` method.
3. Create one rectangle and print it and its area.

Start from the `Point` class on the Circle slide: copy it to the top of your program.

Exercise 2: One Possible Solution



One possible solution, as one complete program:

```
class Point:                                    # Point from the Circle slide
    def __init__(self, x=0.0, y=0.0):           # set up x and y
        self.x, self.y = x, y                   # store both
    def __str__(self):                          # text for print()
        return '(%g, %g)' % (self.x, self.y)    # e.g. (0, 0)

class Rectangle:                                # the rebuilt Rectangle
    def __init__(self, corner, width, height):  # step 1: set up all fields
        self.corner = corner                    # a Point inside the Rectangle
        self.width = width                      # its width
        self.height = height                    # its height
    def __str__(self):                          # step 2: text for print()
        return 'Rectangle at %s, %g x %g' % (self.corner, self.width, self.height)  # the text
    def area(self):                             # step 2: MY area
        return self.width * self.height         # width x height

r = Rectangle(Point(0, 0), 100, 200)            # step 3: make one rectangle
print(r)                                        # -> Rectangle at (0, 0), 100 x 200
print(r.area())                                 # -> 20000
```

Exercise 2 Solution: Line by Line (1/2)



Line	Level	Code	What it does
1	0	`class Point:`	The Point class from the Circle slide.
2	1	`    def __init__(self, x=0.0, y=0.0):`	`__init__`: set up x and y.
3	2	`        self.x, self.y = x, y`	Store both.
4	1	`    def __str__(self):`	`__str__`: text for `print()`.
5	2	`        return '(%g, %g)' % (self.x, self.y)`	Return `(x, y)`.
7	0	`class Rectangle:`	The rebuilt Rectangle.
8	1	`    def __init__(self, corner, width, height):`	Step 1: `__init__` takes the corner, width and height.
9	2	`        self.corner = corner`	Store the corner (a Point inside the Rectangle).
10	2	`        self.width = width`	Store the width.

Line = number on the code box. **Level** = indentation: 0 = at the left edge, 1 = 4 spaces in, 2 = 8, 3 = 12; cont. = continues the line above. Grey rows were explained on an earlier slide.

Exercise 2 Solution: Line by Line (2/2)



Line	Level	Code	What it does
11	**2**	`        self.height = height`	Store the height.
12	**1**	`    def __str__(self):`	Step 2: `__str__`, text for `print()`.
13	**2**	`        return 'Rectangle at %s, %g x %g' % (self.corner, self.width, self.height)`	Return a description; the corner's text comes from Point's `__str__`.
14	**1**	`    def area(self):`	Step 2: method `area`.
15	**2**	`        return self.width * self.height`	Return width times height (`*` means multiply).
17	**0**	`r = Rectangle(Point(0, 0), 100, 200)`	Step 3: make one rectangle at (0, 0), 100 wide, 200 high.
18	**0**	`print(r)`	Print it: `Rectangle at (0, 0), 100 x 200`.
19	**0**	`print(r.area())`	Print its area: `20000`.

Line = number on the code box. **Level** = indentation: 0 = at the left edge, 1 = 4 spaces in, 2 = 8, 3 = 12; cont. = continues the line above. Grey rows were explained on an earlier slide.

Recap



Class	Blueprint. `class Time:`
Object / instance	One thing built from it. `Time()`
Attribute	A named value inside. `t.hour`
Mutable	Attributes can change; aliases see changes.
Copy	`copy.copy` shares inner objects; `deepcopy` does not.
Pure function	Returns a new object, inputs unchanged.
Modifier	Changes the object it receives.
Method	Function inside a class; `self` is the object.
`__init__`, `__str__`	Set up / describe the object.
Overloading	`__add__` gives meaning to `+`.
Polymorphism	One function works on many types.
Interface	What users see; implementation can change.

Further reading

Think Python, Chapters 15, 16, 17 and their exercises.



Questions?

r4ramzan@gmail.com



Thank You!